

The Multi-Tenant Security Manifesto

PS-DOS – CUMULATIVE SECURITY ARCHITECTURE

Every tenant's data, every operator's action, and every privileged credential is contained by a control that does not depend on the application behaving correctly. This briefing documents the full architecture and the eight-round hardening pedigree behind it.

RELEASE	COMMIT	HARDENING ROUNDS	DATE
v1.17.0	3c8ed78	8 (v1.10 → v1.17)	2026-09-16

01 WHY THIS IS BEST-IN-CLASS FOR MULTI-TENANT SAAS

Most multi-tenant SaaS enforces tenant separation in one place — an `WHERE organization_id = ?` the application is trusted to add every time. One missed clause is a cross-tenant breach.

PS-DOS does not rely on the application being correct. Tenant isolation is enforced independently at **four** layers; privileged credentials are cryptographically separated and independently rotatable; every privileged operator action is least-privilege, second-factor gated, time-boxed, and written to a ledger that is immutable at the database engine. A defect in any single control is contained by the others.

THE DESIGN PRINCIPLE

Defense in depth is not a slogan here — it is the file structure. Each guarantee below is enforced by a mechanism that keeps working when the layer above it fails.

02 THE THREAT WE DESIGNED AGAINST

The adversary is not only an external attacker. It is also a well-meaning code change that forgets a scope clause; a phished operator password; a replayed one-time code; a support engineer with more access than the task needs; and a subpoena-grade demand to prove what happened and when.

THREAT	PRIMARY CONTROL
Cross-tenant data access (bug or malice)	Four-layer isolation, incl. database RLS (§ 3)
Phished / stolen operator password	Mandatory TOTP step-up + JIT elevation (§ 6)
Replayed second factor → duplicate session	Atomic single-use consumption (§ 5)
Session-secret rotation bricking MFA	Versioned, decoupled MFA key (§ 4)
Over-broad operator access	Six least-privilege tiers, 3-layer enforcement (§ 6)
Tampering with the record of what happened	Engine-level ledger immutability (§ 7)
Fail-open on infrastructure error	Fail-closed session state machine (§ 8)

03 TENANT ISOLATION — FOUR INDEPENDENT LAYERS

Cross-tenant separation does not rest on any single control. A row created under tenant A is invisible, immutable, and un-creatable from tenant B's context at every layer below — verified by an end-to-end isolation suite.

L1 Application — ORM auto-scope

A Prisma `$extends` extension rewrites every tenant query to filter by `organizationId` and injects it on every create. The scope is applied by the data layer, not by the caller. `runUnscoped` is the single audited exception, reserved for the cross-tenant operator path.

L2 Referential — composite foreign keys

Every intra-tenant relationship is `(organizationId, col) → parent(organizationId, id)`. The database rejects a row that references another tenant's parent — a cross-tenant reference is not an authorization failure, it is a constraint violation.

L3 Database — Row-Level Security (RLS)

Every tenant transaction runs `SET LOCAL ROLE a2r_app + SET LOCAL app.current_org = <id>`. `tenant_isolation` policies on 28 tenant tables filter every read and write to the current org; a hard `rls_deny_app` policy on the 9 identity tables blocks the runtime role outright. The role `a2r_app` is `NOLOGIN`, non-superuser, and `rolbypassrls = false` — it cannot turn RLS off. A connection that forgets to set the GUC sees

zero rows
, not all rows.

L4 Environment — deployment isolation

Build, server boot, and Prisma-client instantiation all hard-fail if a non-production deployment's database URL resolves to the production project. A preview build cannot accidentally point at production data.

RLS is enforced on staging and applied plus structurally verified on production (read-only acceptance check, zero DML). `FORCE ROW LEVEL SECURITY` is the post-soak hardening step.

PS-DOS holds three classes of secret material — bearer tokens, the operator MFA secret, and SSO client secrets. Each has a storage rule that assumes the database will one day be read by someone who should not have it.

MFA secret — dedicated, versioned key, decoupled from the session secret

The operator MFA (TOTP) secret is sealed with **AES-256-GCM** under `MFA_ENCRYPTION_KEY` — a dedicated key that is **fully decoupled from NEXTAUTH_SECRET**. The application refuses to boot in production if the two are equal. Rotating the session-signing secret therefore has zero effect on enrolled authenticators.

Ciphertext is self-describing, so the decrypt path always knows which key produced it:

<code>v2.<keyVersion>.<iv>.<authTag>.<ciphertext></code>	current format (scrypt KDF, AES-256-GCM)
<code>v1.<iv>.<authTag>.<ciphertext></code>	legacy format — still decrypts

Zero-downtime rotation: a new key is introduced as `MFA_ENCRYPTION_KEY` with `MFA_ENCRYPTION_KEY_VERSION` incremented; the prior key is retained decrypt-only as `MFA_ENCRYPTION_KEY_V<n>`. A secret sealed under an older key version is transparently **re-sealed under the current key on its next successful verification**. No migration job, no downtime, no forced re-enrollment.

Bearer tokens — hash-at-rest, plaintext only in transit

JIT-elevation tokens, impersonation grants, and tenant API keys are stored as sha256 digests only. The plaintext exists solely in the `httpOnly` cookie or the one-time creation response and is never persisted. Comparison is constant-time. SSO client secrets are AES-256-GCM encrypted at rest.

05 ATOMIC REPLAY PROTECTION

A read-then-write check for "has this code been used?" has a time-of-check / time-of-use gap: two concurrent requests both read "unused" and both proceed. PS-DOS closes that gap at the database, not in application logic.

TOTP — a single conditional UPDATE

The anti-replay high-water mark advances only if the presented code's time-step is strictly newer, in one atomic statement:

```
UPDATE operator_mfa
  SET last_step_counter = :candidate
  WHERE user_id = :id
     AND (last_step_counter IS NULL OR last_step_counter < :candidate);
-- verification succeeds only if exactly one row was updated
```

Two concurrent requests presenting the same valid code: the mark advances once, exactly one request updates a row and succeeds, the other updates zero rows and is rejected as replayed.

Recovery codes — SELECT ... FOR UPDATE row lock

Recovery-code consumption runs inside a transaction that takes a SELECT ... FOR UPDATE row lock on the code before marking it spent. A concurrent consumer blocks on the lock, then reads the code as already used.

Recovery codes are high-entropy random values stored as SHA-256 — a password KDF is neither needed nor appropriate for them.

VERIFIED

Both paths have dedicated concurrency tests that fire parallel requests and assert exactly one winner — "atomic anti-replay — two concurrent requests" and "atomic consumption — recovery code race".

06 THREE-LAYER OPERATOR RBAC — SIX TIERS + GUEST OBSERVERS

The internal operator console is not reachable with any tenant membership. It requires an explicit, attributed, revocable `staff_grants` entitlement, and that grant carries exactly one of six organizational roles.

OPERATOR TIER	REMIT
Super Admin / Owner	Full access — the only tier that may purge a tenant, manage staff grants, change roles, or manage API keys.
Provisioning	Tenant onboarding and lifecycle, SSO setup, ingestion configuration.
Support / Troubleshooting	Diagnostics, platform health, read-only impersonation, audit view.
Auditor / Compliance	Read-only — audit ledger, elevation history, contract records, tenant export.
Billing / Finance	Contract tiers, seat counts, subscription state.
Viewer	Restricted read-only — platform pulse and telemetry only.

Enforced in three independent layers

- A Edge middleware**
A route guard redirects a role away from any `/ops` sub-route its capability set does not include — before the page code runs.
- B Server page guard**
Every `/ops/*` page calls `requireOpsCapability(cap)`; a missing capability redirects to the role's landing route.
- C Server action guard**
Every mutating operator action calls `requireElevatedOps(cap)` and returns `ROLE_FORBIDDEN` before touching data — independent of what the UI rendered.

Just-In-Time elevation — no standing privilege

A role grants *eligibility*, not standing access. Every mutating operator action **also** requires a live JIT elevation obtained in one request with: a stated reason (≥ 10 chars), a fresh `bcrypt` password re-verification, **and** a valid TOTP code or single-use recovery code. The elevation is bound to the session epoch, auto-expires (default 30 min, hard cap ≤ 240), and there is at most one live elevation per operator. A password change or global sign-out kills every active elevation instantly. Changing a role is itself an audit-preserving re-grant, gated on Super Admin + elevation — and an operator can never change their own role.

Isolated guest observers

On the tenant side, the `VIEWER` tier is strict read-only: whole-org read of the portfolio and the SteerCo board, zero edit authority anywhere, and every financial figure scrubbed server-side to the `restricted` masking tier. An account at this tier is structurally walled off from the operator console and from tenant administration — verified by a dedicated end-to-end suite in which every `/ops/*` and `/admin` route redirects it away.

07 ENGINE-LEVEL AUDIT IMMUTABILITY

Every operator action that touches a tenant, and every privilege elevation, is written to a hash-chained ledger. The ledger is immutable **at the PostgreSQL engine**, not by application convention:

- The runtime role `a2r_app` holds **no UPDATE or DELETE privilege** on `immutable_audit_ledger` (verified via `information_schema.role_table_grants`).
- A `BEFORE UPDATE / DELETE / TRUNCATE` trigger rejects every caller — including a superuser — unless a deliberate, transaction-local GUC is set that the runtime role can never set. That opt-in exists only for lawful data-subject erasure.
- Integrity is continuously verifiable: each entry's hash chains to its predecessor, and a verification routine walks the chain.

Tenant-side, the same hash-chained ledger records governance-relevant activity per organization and is surfaced read-only at `/admin/audit-log`.

08 FAIL-CLOSED SESSION INTEGRITY

Every authenticated request re-derives session state from the database: the `sessionVersion` epoch, `passwordChangedAt`, and the forced-rotation flag. The state machine has one bias — **any uncertainty fails closed**:

CONDITION	RESULT
Database error resolving session state	REVOKED — user-less session
Lookup exceeds <code>SESSION_LOOKUP_TIMEOUT_MS</code>	REVOKED
User row missing / deleted	REVOKED
Token epoch $\neq$ account epoch	REVOKED
<code>mustChangePassword</code> set	<code>PENDING_PASSWORD_CHANGE</code> — only the change-password route

A password change atomically writes the new hash, stamps `passwordChangedAt`, increments `sessionVersion` (an all-device logout), and mints exactly one fresh token for the acting device. There is no fail-open path.

Surface hardening

- **Rate limiting** fails *closed* in production when a shared backend is configured and unreachable — deny and alert, never silent degradation.
- The **public readiness probe** returns only up/down; database dependency, latency, and error type require an internal token.
- **Error sanitization** — no raw backend error (Prisma exception, stack, driver message) reaches a client; statically enforced across every API route.
- **Operational CLIs** never accept a password as a command-line argument, and refuse an unconfirmed production write; admin-forced resets set `mustChangePassword` and invalidate sessions.

09 THE EIGHT-AUDIT PEDIGREE

PS-DOS reached v1.17.0 through eight successive rounds of adversarial AI-assisted self-audit and hardening.

Each round ran the same discipline: an independent review by an AI audit agent (ChatGPT) producing prioritized, adversarial findings; implementation and regression tests by a second AI agent (Claude), with every database migration rehearsed BEGIN ... ROLLBACK before application; a verification gate the round closed only on a full green suite; and a release tag cut at the **exact immutable commit** deployed to production.

ROUND	RELEASE	HARDENING FOCUS
1	v1.10.0	Mass-assignment lockdown (strictObject); all-device sign-out; strict cookies.
2	v1.11.0	Exact-decimal money columns; FK onDelete: Restrict on the operator-grant chain.
3	v1.12.0	Production RLS cutover; identity-table hard denial; a2r_app set NOLOGIN.
4	v1.13.0	Engine-level audit-ledger immutability; composite-FK closure; global break-glass removed.
5	v1.14.0	Exact-decimal arithmetic through the calc engine; JIT elevation password step-up + session binding; test-rig production isolation.
6	v1.15.0-.2	Rate-limiter fail-closed; mandatory operator TOTP MFA; MFA key separation; atomic replay protection; readiness-probe + CLI hardening.
7	v1.16.0	Six-tier operator RBAC with a three-layer capability matrix; Role & Access console; strict read-only tenant Viewer tier + guest observers.
8	v1.17.0	Unified data-driven RBAC nav matrix reconciled against real per-project edit authority; Write-Gate Alignment across every per-project editor; explicit Persona Preview banner replacing a legacy, disconnected preview menu.

COMPLIANCE ALIGNMENT – STATED PRECISELY

The platform is **designed and operated in alignment with SOC 1 and SOC 2 control objectives**: logical access control, change management, segregation of duties, and a tamper-evident audit trail for financially-relevant activity. This alignment is demonstrated by the evidence artifacts in the repository — the requirements traceability matrix, the test-coverage report, and the hash-chained ledger. **No third-party SOC 1 or SOC 2 attestation has been performed**, and nothing in this document should be read as claiming one. Formal attestation is a business decision; the control-design work and evidence are already in place to support it.

10 VERIFICATION EVIDENCE (V1.17.0)

TYPE CHECK	LINT	UNIT + DB	END-TO-END
0	0 / 0	695	65
BUILD	PROD RLS CHECK	PROD HEALTH	MIGRATIONS

CONTROL	REPRESENTATIVE AUTOMATED PROOF
Tenant isolation (all 4 layers)	tenant-isolation, rls-policies, env-isolation; e2e Suite O; SQL RLS smoke (10-check) + read-only prod acceptance
MFA key separation & rotation	secret-box.test.ts — versioned key, rotation with old-key ciphertext still decrypting, legacy v1 decrypt
Atomic replay protection	operator-mfa.test.ts — two concurrent requests, recovery-code race
3-layer operator RBAC	operator-roles.test.ts (13), operator-role-grants.test.ts (7); e2e Suites P, Q
Ledger immutability	ledger-immutability, ledger-concurrency; e2e Suite F
Fail-closed session state	session-state.test.ts (29), password-rotation-flow.test.ts; e2e Suite N

11 RESIDUAL ITEMS — DISCLOSED

ITEM	STATUS
WebAuthn / passkeys for phishing-resistant AAL2	Tracked — slots in at the verifySecondFactor seam; TOTP is the interim second factor.
Legacy operator accounts not yet MFA-enrolled	Operator action — the hard cut-over means they must enroll before they can elevate.
FORCE ROW LEVEL SECURITY on production	Post-soak — RLS is applied and enforced; FORCE follows the soak window.
Dedicated a2r_ops database role for the cross-tenant admin path	Phase-3 residual — currently runs as postgres via audited runUnscoped.
Third-party SOC 1 / SOC 2 attestation	Not started — control-aligned; attestation is a business decision.